An Efficient Bit-level Sparse MAC-accelerated Architecture with SW/HW Co-design on FPGA

Chenming Zhang, Lei Gong*, Chao Wang* and Xuehai Zhou
School of Computer Science and Technology, Suzhou Institute for Advanced Research
University of Science and Technology of China

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133249

***Abstract*—Exploring bit-level sparsity in the MAC process has been proven to be an important method for improving the efficiency of neural network feedforward processing. The reconfigurable platform offers possibilities for identifying the bit-level unstructured redundancy during inference with different DNN models. Researchers noticed significant progress in value-aware accelerators on ASICs, yet we are concerned about the few studies on FPGAs. This paper observed the limitations of implementing bit-level sparsity optimizations using FPGA and proposed a software/architecture co-design solution. Specifically, by introducing LUT-friendly encoding with adaptable granularity and hardware structure supporting multiplication time uncertainty, we achieved a better trade-off between potential redundancy and accuracy with compatibility and scalability. Experiments show that under accurate calculation, PEs are up to 2.2× smaller than bit-parallel ones, and our design boosts performance by 1.04× to 1.74× and 1.40× to 2.79× over bit-parallel and Booth-based designs, respectively.**

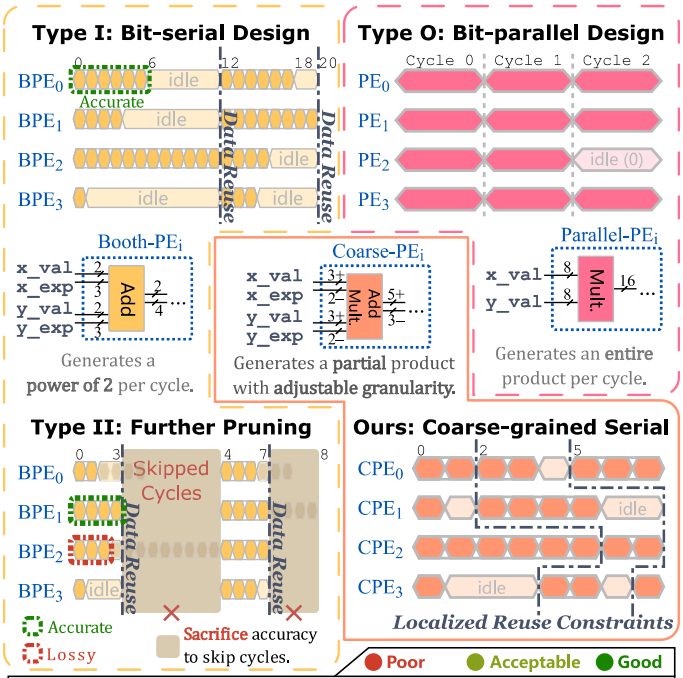
***Index Terms*—Bit-level sparsity, multiply and accumulate, SW/HW co-design, FPGA, look-up table.**

I. Introduction

The flourishing of deep neural networks (DNNs) has been engendering a rapidly escalating hardware demand for computation and storage, posing new challenges to model performance and efficiency during inference. Together with general-purpose CPUs and GPUs, researchers are also vigorously prompting inference acceleration based on ASIC [1]–[6] and FPGA [7]–[10] platforms. Matrix multiplication and convolution dominate over 90% of DNNs [11], with their computations almost entirely composed of multiply-and-accumulate (MAC) operations. The performance of MACs induces crucial impacts on accelerators.

After appropriate training and quantization, the activation or weight data are more likely to be equal to or close to zero [12]. Bit-level sparsity optimization attempts to skip "0" bits within a binary value, Previous works [13]–[17] have given us a convincing demonstration that the distribution characteristics of non-zero values, in addition to strictly zero values, is also plausible for acceleration. The plausibility in calculation turns into reality through bit-serial multipliers based on Booth encoding [2], [4], [7]. Booth encoding eliminates consecutive 0s and 1s, allowing serial multiplication faster, thus surpassing bit-parallel multipliers. Preprocessing methods like Booth encoding, can effectively reveal the bit-sparsity of operands and cope with varying data characteristics among different data sets.

On the one hand, we have perceived that in the former designs, the awareness of bit-sparsity, the certainty of multiplication time, and the accuracy of results can be satisfied at most two out of the three. To preserve the accuracy of serial multipliers, one must tolerate the unpredictable number



Solution(s)	Type O	Type I	Type II	Ours
Bit-Sparsity	None	Max(with optimal booth encoding*)		Consecutive 0/1
Codec Cost	None	High		Low
Accuracy	Loseless	Loseless	Lossy	Loseless
Utilization	High	Low	Relatively High	Relatively High
Bitwidth	Fixed	Variable	Variable	Adaptable

*Optimal Booth Encoding: See Section II-B.

Fig. 1. Comparison of temporal utilization. Multipliers change from bit-parallel (Type O) to bit-serial (Type I/II) after introducing bit-sparsity awareness. The certainty of multiplication time (Type II) and the accuracy of results (Type I) cannot be satisfied simultaneously.

This work is supported in part by the National Key R&D Program of China under Grants 2022YFB4501600 and 2022YFB4501603, in part by the National Natural Science Foundation of China under Grants 61976200 and 62172380, in part by Jiangsu Provincial Natural Science Foundation under Grant BK20241818, in part by Youth Innovation Promotion Association CAS under Grant Y2021121, in part by USTC Research Funds of the Double First-Class Initiative under Grant YD2150002011. *Lei Gong*, *Chao Wang* and Xuehai Zhou are corresponding authors. Email: {leigong0203, cswang, xhzhou}@ustc.edu.cn.

of cycles for non-zero products, leading to the issue of load imbalance [1]–[4]. The derived methods [7], [10], [14], [18], [19] aim to evenly allocate the workload by pruning some of the non-zero bits or trits. However, the pruning refers to secondary quantization, which in turn lowers the performance of models. If it is possible to perform a bit-level customization in compliance with the data characteristics of the target workload, A better trade-off would come to pass for the target model.

On the other hand, innate advantages for customizing bit-sparsity acceleration are in line with the bit-level reconfigurability of FPGAs, anyway there are only a few studies [7], [8], [10]. We are of the opinion that this is caused by the poor resource utilization imputable to the primitive structure of look-up tables (LUTs). Extremely high parallelism in bit transmission, including fan-ins and fan-outs, is typical for bit-serial multiplier logic, which is difficult to deploy efficiently via LUTs. It is adequate to negate the performance of dynamically identifying bit-sparsity at the level of multipliers alone.

Following the observations above, we advanced bit-serial multipliers to a more adaptable co-design solution for encoding and architecture, the three aspects described above. In section III, a compute-oriented, LUT-friendly, and granularity-adjustable encoding is introduced while preserving accuracy. In Section IV's architecture, at the multiplier level, processing elements are coordinated with the encoding. Then, at the cluster level, load-balancing strategies are available for multiplication time uncertainty. As a foundation, we revise approaches and challenges for bit-level sparsity awareness in section II.

II. Background and Motivation

A. Bit-level Sparsity Acceleration

Within DNN inference scenarios, integers and fixed-point numbers are appreciated for their hardware-friendliness, as floating-point numbers' exponential capability is too excessive. The non-uniform distribution of the integers results in less effective information than n bits contained in an n-bit number, exhibiting redundancy within the data.

Per layer precision [20], [21] is a form of bit-level sparsity, since values with smaller bit widths are equivalent to larger ones with bit-sparsity at higher bits. Such quantization method has spawned accelerators that support multiple precisions [22]–[24], which combine their small bit-width multipliers through reconfigurable logic, as shown in Fig. 2(a), belongs to type O in Fig. 1. The layer-wise analysis only reveals the differences in importance between layers, without analyzing the non-uniformity within a layer. To exploit redundancy further, the multipliers should be value-aware at run-time. Specifically, they dynamically adjust the spatial or temporal cost based on the precision of the operands. The spatial redundancy of bit-parallel multipliers, shown in Fig. 2(a), should first be converted into the temporal redundancy of bit-serial multipliers, shown in Fig. 2(b), and only then can it possibly be removed.

However, this approach does not conform to the paradigm of most accelerators. In General, data are arranged regularly, and each value equally possesses the same precision. The schedule for transmission and calculation is predictable across space and time, thus avoiding catastrophic data transmission. Once bit-level sparsity is involved, unpredictable dynamic precisions emerge among different values, leading to unstructured sparsity. A randomly varying temporal cost for each multiplication takes place, and a load imbalance issue occurs under the constraints of data reuse, shown in Fig. 1 type I. It not only negatively impacts inference latency, but exacerbates load imbalance with enhanced reuse and stronger communication constraints, which is detrimental to design scalability. A straightforward way is to directly address the bottlenecks that provoke load imbalance by pruning some lower bits, as shown in Fig. 2(c) and type II in Fig. 1, but it inevitably impairs the performance of the target model. Forcing models to adapt to the accelerator would weaken its compatibility with models into the bargain.

Introducing a value-aware multiplier provides feasibility for revealing bit-sparsity and then acceleration. Unfortunately, such designs have created a conflict between load balancing and lossless computing, hindering the speed and quality of inference. In brief, the awareness of bit-sparsity, the certainty of multiplication time, and the accuracy of the results cannot be achieved at the same time.

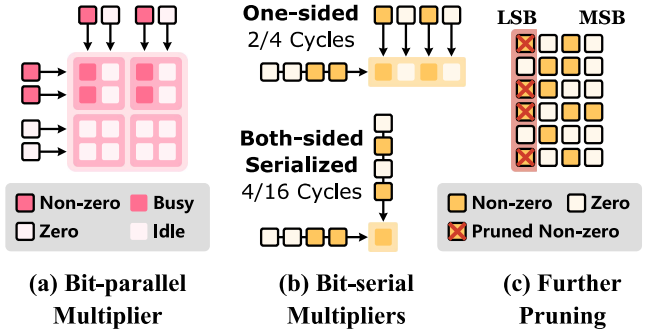


Fig. 2. (a) A reconfigurable parallel multiplier. (b) One-sided and both-sided serial multipliers. (c) Pruning to mitigate multiplication time uncertainty in serial multipliers.

B. Booth Encoding for Preprocessing

A value-aware multiplier design is built on or similar to the serial Booth multiplier. As a means of pre-processing, Booth encoding appears to have sufficient revelation to bit-sparsity. We discuss whether the pre-processing plays a critical role in the conflict described in section II-A.

$$x = \mathrm{val}(B_x) = \sum_{s=0}^{n-1} b_s 2^s \tag{1}$$

Equation (1) shows how the coefficient sequence $B_x = (b_{n-1}, b_{n-2}, ..., b_0)$ maps to the original value x. As shown in Fig. 3(a), the standard integer representation enables only two states $b \in \{0, 1\}$ for each coefficient, while $\{0, -1\}$

for significands. In radix-2 or radix-4 Booth encoding, each coefficient equally enables three states $b \in \{-1, 0, 1\}$. For an n-bit value, since $\left|\{-1, 0, 1\}^{n+1}\right| \gg |\{0, 1\}^n|$, there must be multiple Booth encodings mapping to the same value. It allows us to select an encoding with the smallest non-zero coefficient set, which we call the "optimal Booth encoding".

The optimal Booth encoding performs a precise analysis for the bit-sparsity down to each bit. It exhaustively exposes redundancies arising from non-uniformity. Booth encoding, or any compute-oriented preprocessing method, is usually required to accomplish very quickly. In contrast, in the inference scenario there are lower thresholds for preprocessing, because the encoding overhead is amortized by repeated multiplications.

In opposition, it's expensive to extend the bit width from fixed to variable. Serial multipliers require splitting the coefficient sequence, a process that results in loss of alignment information. We have to append some positional bits, which causes on-chip communication overhead, and nothing to say adding a third state to the coefficients, As for the bit-sparsity analysis down to each bit, it is sometimes an excessive treatment that is of no good or even harmful. For lower bits of larger values, although relatively unimportant, they contain sufficient information, so any preprocessing struggles to analyze redundancy from them. The fluctuation of non-zero term set size is ralated to such excessiveness, as shown in Fig. 3(b), along with underutilization problems under reuse constraints.

C. Bit-serial Multipliers on FPGA

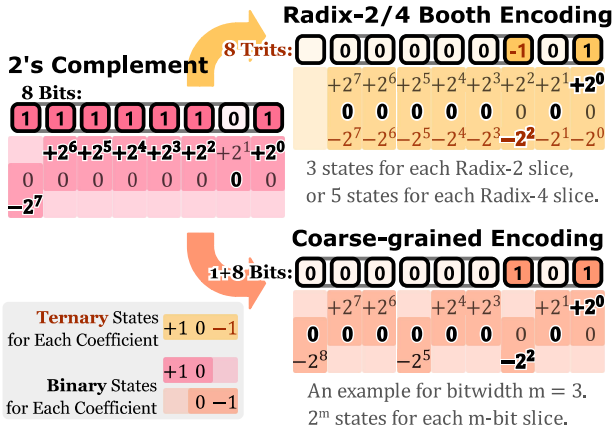
Massive interconnections and intermediate results, as well as simple gates and selectors, are common features of bit-serial multipliers on ASIC platforms. 5 to 6-input LUTs are considered the most suitable choice in common [25], yet when dealing with simple parallel bit operations they are more susceptible to I/O bottlenecks. An intolerable waste arises that a LUT6's functionality is at least the same as a 6-input ROM, but is likely to implement a few basic gates.

The Radix-4 Booth encoding outputs 5 states for each 2 bits instead of 4, inflating the number of bits required for each coefficient to 150%. This leads to a decrease in the information density among on-chip buffers and multiplications. Consequently, we need a preprocessing method that is more friendly to look-up tables. The reconfigurability allows adjusting the granularity of bit-sparsity revelation and switch granularity between target workloads, provides us the opportunity to alleviate the excessive treatment.

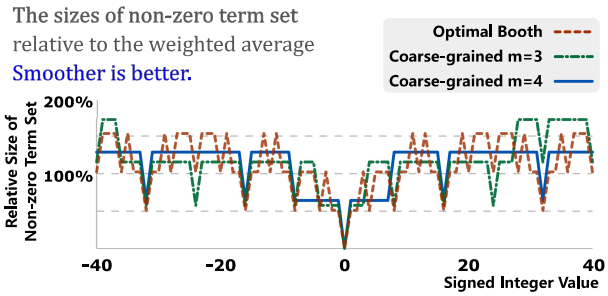
III. Compute-Oriented Codec Method

The preprocessing that reveals bit-sparsity should have the following properties, as specified in section II-A.

- (a) A consistent format for each generated term (or slice).
- (b) The negative correlation between the size of the non-zero term set and the proportion of the corresponding value in a dataset.



(a) Treating Negative Values



(b) Variation of the Size of Non-zero Term Set

Fig. 3. The proposed coarse-grained encoding: (a) introduces fewer additional states, is more LUT-friendly, and (b) generates fewer and smoother numbers of non-zero terms, and simplifies the scheduling.

Furthermore, section II-B and II-C put more potential improvement forward.

- (c) An adaptable granularity for bit-sparsity analysis.
- (d) A higher information density in all terms generated.

An obvious and low-cost approach is to divide the binary representation into slices of length m. For signed integers, it yields one signed term and several unsigned terms, which complicates the preservation of property (a). Alternatively, it can be deduced from property (b) that each term should be signed, otherwise negative values with smaller absolute values cannot be represented by fewer non-zero terms. Simply attaching a duplicated significand to each term would result in redundancy. Hence we consider an approach of embedding rather than attaching sign information.

- ***Serialize.*** Divide the n-bit value into m-bit slices. A term consists of a coefficient $B_{x,i}$ and a shift $s_{x,i} = im$ with its weight $w_{x,i} = 2^{im}$. Perform sign extension on the most significant term if needed.
- ***Embed.*** Flip the state definition of the MSB in each coefficient $B_{x,i}$, i.e., $B_{x,i}[m-1] = b_{im+m-1} \in \{0, -1\}$, without appending to or changing present bits.
- ***Regulate.*** Add $B_{x,i}[m-1]$ to $B_{x,i+1}$ from lower to higher terms. Carry over to the next higher term if an overflow occurs.

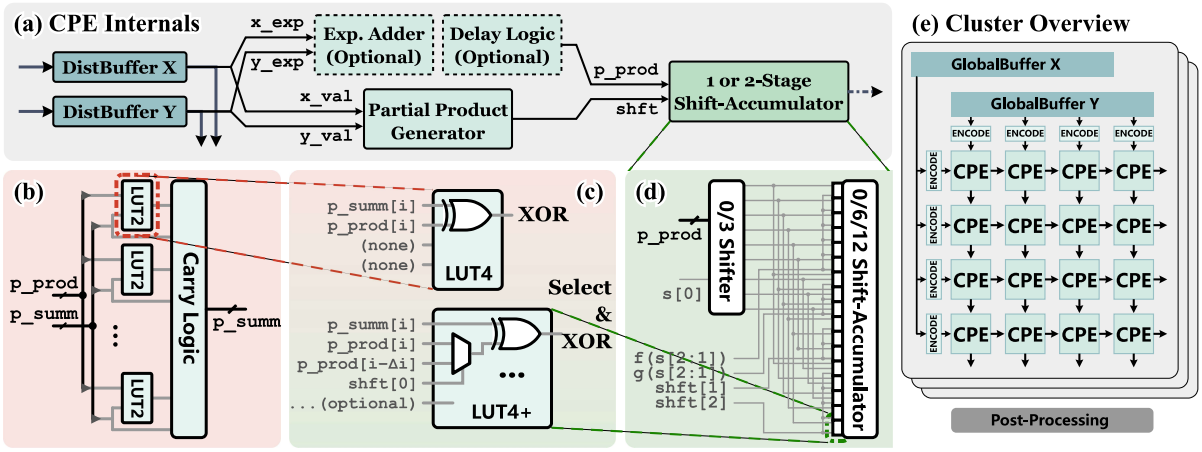


Fig. 4. (a) Internal components of a PE. (b) An adder with fast carry chains. (c) Merging shift logic into LUTs. (d) An example shift-accumulator for Xilinx FPGA. (e) Clusters composed of PEs in (a).

The preprocessing procedure utilizes only the existing bits to represent sign information with no extra bits appending to each term, as shown in Fig. 3(a). Note that the extremes of signed integers (the largest $1/2^{m+1}$ in the range) require an extra bit at the most significant term due to overflow, i.e., at most $\lceil (n+1)/m \rceil$ non-zero terms. All the generated coefficients are represented as signed or 2's complement unsigned integers, with $\sum \mathrm{val}(B_{x,i}) \times w_i = \mathrm{val}(B_x)$. Therefore, we can sum the Kronecker product of the two term sets to complete a multiplication, as shown in (1), thus satisfying property (a).

$$xy = \sum_{i,j} \mathrm{val}(B_{x,i})\mathrm{val}(B_{y,j}) w_{x,i} w_{y,j} \tag{1}$$

The lengths m of partitions of the operands on both sides can vary as integers between 2 and n, modifying the range of coefficients to satisfy property (c). The two bit widths of the operands with a sum of less than 6 is unsuitable for fully utilizing the 6 or more-input LUTs [26]. Accordingly, m should be no less than 3. m_x and m_y for the two operands x and y are preferably be the same or have a larger greatest common divisor, as detailed in section IV-A. For the coarse-grained encoding with bit width $n = 8$ and granularity $m \in \{3, 4\}$, Fig. 3(b) shows the correlation that the size of the non-zero term set varies with the value, compared to the optimal Booth encoding.

Please note that the proposed encoding, while similar to Booth encoding, has essential differences. A coarser-grained encoding, which reduces the mapping space and implies a weaker analysis of redundancy, admittedly loses some revelation in bit-sparsity. Instead, it alleviates the excessive treatment problem in Radix-2 or 4 Booth encoding while maintaining property (b). Moreover, it allows more potential in optimizing scheduling overhead, as Fig. 3(b) also shows that The sizes of the sets no longer fluctuate dramatically. Radix-2^m Booth encoding always defines an additional coefficient state $+2^m$ to avoid carry-overs, and enables the parallelism of multiple-term generations. Even so, one is more concerned with the overall throughput rather than the latency of a single multiplication in intensive and regular operations. It follows that the parallelism of the preprocessing is irrelevant. Such coarse-grained encoding sacrifices latency, in exchange for more compliance with property (d), namely, a more LUT-friendly format for transmission and computation.

IV. Hardware Architecture

A. Processing Unit Design

According to the serialization, PEs are designed to generate partial products first. In each cycle, the distributed buffers inside provide a pair of terms $(B_{x,i}, s_{x,i})$ and $(B_{y,j}, s_{y,j})$. The two coefficients $B_{x,i}$ and $B_{y,j}$ are input to the parallel multiplier, generating a partial product with a bit width of up to m_x+m_y, meanwhile the adder generates the shift $s_{x,i}+s_{y,j}$. Fig. 4(a) shows the execution order for partial products inside a PE. Over multiple cycles, a unit reads and combines from the two buffers inside, and traverses the Descartes product of the non-zero term sets of two operands to make up a complete multiplication. An adder for shifts is not necessary when the number of possible shifts is too small. The partial product and its shift are expected to arrive at the next stage simultaneously, thus delay logic may be required.

In the next stage, the shift of the partial product is variable and unpredictable. LUTs are not good at implementing logic with lots of inputs and outputs, including a parallel-out adjustable shifter and counting buckets. For this reason, we reduce the number of shift operations and the shift is simplified. The number of possible shifts is a decisive factor affecting the resource occupancy of parallel-out shifters. To prevent shifting with more than two levels of LUTs, a larger greatest common divisor for the operands' shifts is suggested. For instance, with two 8-bit operands and their encoding bit width $m_x = m_y = 3$, there are only 5 different possible shifts ($\{0, 3, 6, 9, 12\}$). If $m_x = 3, m_y = 4$, it not only does not decrease but increases to at least 6 ($\{0, 3, 6, 4, 7, 10\}$).

Due to good properties of addition, We only need one accumulator for each inner product. As shown in Fig. 4(b), if

a separate adder is implemented, the relatively simple addition becomes even easier with the help of carry chains. Each LUT is used only as an XOR gate, leaving redundancy in functionality. The spare inputs enable them to perform limited shift operations, as shown in Fig. 4(c). Therefore we merge part of the shifter into the accumulator while keeping another part at a separate level.

Fig. 4(d) the mapping of shift with sign extension to two-level LUT6s on shift granularity $\gcd(m_x, m_y) = 3$ with Xilinx FPGAs. For $\gcd = 4$ and above it comes to be simpler. In contrast, the optimal Booth encoding only guarantees that the shift distance is a multiple of 2 bits.The area of the partial multipliers is positively correlated with the encoding bit width m, while the shift-accumulators are almost unaffected.

B. Array Organization and Communication

As shown in Fig. 4(e), we organize the PEs into output-stationary arrays. In a systolic array, all PEs logically update their states synchronously, and each PE only obtains the most significant information from its neighbors (horizontally, vertically, or diagonally, etc.). After introducing uncertainty in multiplication time, a simple approach is to wait for all PEs in the array to complete their multiplications or inner products. This is absolutely safe but results in poor temporal utilization, as shown in Fig. 5(a) belonging to Fig. 1 type I.

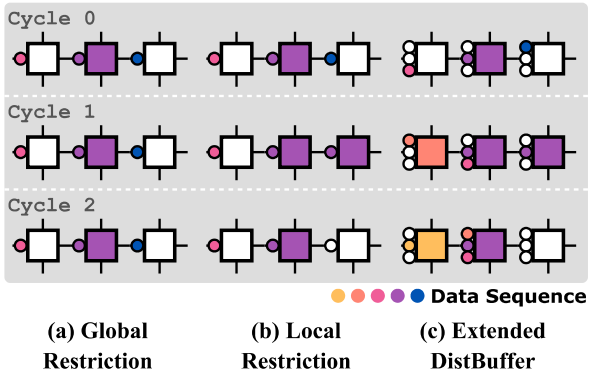


Fig. 5. Assuming that the middle PE encounters a sudden heavy load. (a) Under a global constraint. (b) Downstream PE releases in advance. (c) Upstream PE obtains data in advance.

Here, we adopt some more aggressive but safe communication strategies. Each PE has a set of registers for the operands in both directions X and Y. On this basis, we add a 1-bit connection between the originally connected PEs, allowing the neighbors to be aware of each other's status and decide the data transmission occasions on their own. In other words, the read and write of a buffer are no longer performed simultaneously and the restriction of global synchronization is also lifted, as shown in Fig. 5(b). The sudden occurrence of heavy loads may still hinder transmission, so there is room for further relaxation of communication constraints. One set of registers is feasible be extended to two or more sets, as shown in Fig. 5(c), where local congestion upstream is patched. Furthermore, the extended capacity makes it possible for the time uncertainties of successive multiplications within an inner product to cancel each other out. A larger buffer capacity conduces better temporal utilization, but capped to 100%, hence there is a trade-off between buffer capacity and temporal utilization.

C. Model Compatiability

For the repeatedly reused weights in a model, the encoders for them can be greatly simplified to serializers by performing the *Regulate* step in section III offline in advance. It doesn't contribute to any storage overhead, since the coarse-grained encoding doesn't append any bits to the coefficients, and remains orthogonal to data-level and bit-plane compressions.

The calculation is faster for inputs with fewer non-zero terms. The accelerator supports per-data bit-sparsity, so it naturally supports per-layer precision. Secondary float-point-like quantizations are also available for further speed-up, though the proposed architecture aims to avoid it. The performance under different granularity options can be estimated by calculating the product of the average number of non-zero terms on each layer through software means. Section V-B compares the difference between the estimated and the real.

We do not recommend practices such as combining four 6-bit units into a 12-bit unit, as this would lead to an imbalance load between the four units. Communication strategies in section IV-B are designed for local random fluctuations in bit-sparsity and is powerless against load imbalance per line or column. But in convolution operation, the spatial correlation of images and the im2col algorithm ensure that the data distribution of adjacent windows does not have significant differences.

V. Evaluation Results

A. MAC Units Implementation

In the proposed architecture, small parallel multipliers use the Booth multiplier implementation [27] . For comparison, an accelerator with fixed precision and no additional components for serialization are provided. Our implementation is based on Vivado 2023.1 and Xilinx UltraScale+ series' xczu9eg-ffvb1156-2-e device, running at 400MHz, as shown in Table I. Each PE has four sets of 2-bit control signals, 6-bit address lines, and 6-bit data lines for exchanging data with upstream and downstream. The encoding granularity $m = 2$ is definitely weaker than $m = 3$, so it is not shown. The mapping is also valid for the 6, 7, and UltraScale series with the same LUT6_2 components and similar CLB structure.

In the instantiation of the distributed buffer, for the encoding granularity $m \in 3, 4, 5$, one 32×6-bit simple dual-port RAM provides just enough bit width to cache the terms and length information of the encoding. Since the two operands in an inner product are aligned in the two distributed RAMs, the higher 3-bit read addresses are shared, and the write address reuses the read address upstream. Each unit only needs to generate 7 bits of addresses and 2 bits of control signals per cycle, so as to realize the state machine using only one additional CLB. In such implementation, the options of

TABLE I
IMPLEMENTATION FOR COARSE-GRAINED PES AND ARRAYS

Encoding Granularity	RAM and Controller	Multiplier	Adder	Shifter	Accumulator	Total	Number of Units	Overall LUT Utilization
	(CLBs per Unit)							
$m_x = m_y = 3$	2 (1+1)	0.75*	0.25*	0.625*	3.375*	7	3456	75.6%
$m_x = m_y = 4$	2 (1+1)	1.75*	0.25*	0.625*	3.375*	8	3072	75.9%
$m_x = m_y = 5$	1.875 (1+0.875)	4.5*	0	0	3.375*	10	2496	74.9%
None (8-bit)	N/A					18	1456	73.8%

*Shares SLICEs.

$m_x = 3, m_y = 6$ or $m_x = 4, m_y = 8$ has almost no practical value in reducing control overhead.

For the target models, due to the similarity between bit-level and data-level sparsity, the offline load balancing [28] is applied. The differential convolution [2] is also applied if the boost is positive. We choose ResNet-18 and VDSR quantized to 8-bit for comparison with a relatively evident difference in data distribution. Per-layer precision is banned for generality and conciseness. Fig. 6(a) shows the MAC per second difference of the bit-parallel design in each convolutional layer. VDSR, with stronger bit-sparsity, performs significantly better than ResNet-18, and their advantageous granularity m are different. VDSR achieves an average of 1.74× at $m = 4$, while the lowest is 1.04×, with some layers performing even worse than the bit-parallel design. This is reasonable because without any information redundancy, the theoretical speedup is 0.4×, conversely it reaches up to 2.2× at applying per layer precision. The unsatisfactory result with $m = 3$ indicates that the redundant analysis of the mantissa-like LSBs is usually excessive.

Our implementation doesn't make any special treatment for zeros. In other words, multiplications involved with zeros are the same as non-zero ones and take one cycle. For highly sparse models, a data-level sparsity optimization is better. The unstructured bit-level sparsity is relatively mild and is hardly worth any compression representation. Besides, optimization for bit-serial multipliers may be diluted by the logic introduced at the data level, i.e., too sparse to optimize.

B. Communication Strategies Comparison

If a PE does not perform multiplication in a cycle, or performs one results in "0", we call it idle, or macroscopically temporal underutilization. The proportion of idle time is equivalent to the ratio of actual cycles to the estimated cycles based on the product of the average size of the non-zero term set.

Fig. 6(b) shows the variation in temporal utilization rate under the joint influence of encoding granularity and the size of distributed buffers. For ResNet-18, compared to the naive strategy of a low-power design [4], the improvement in the utilization rate brought by relaxing communication constraints ranges from 1.40× to 2.79×. It only needs two extra CLBs, since the distributed buffers are easier to fully exploit the functionality of LUTs. A LUTRAM with a depth of 32 and capable of holding eight integers is more than enough. The potential improvement is no more than 16%, while adding

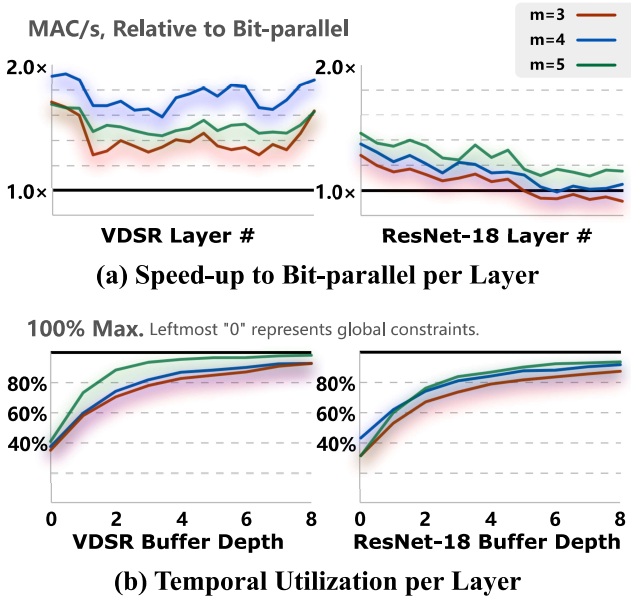


Fig. 6. (a) The advantageous encoding granularity m is larger. This indicates that MSBs mainly cause redundancy. (b) Buffer depth 8 is more than enough.

a SLICE would increase the area cost by 5% to 7%, thus one LUTRAM is already at the sweet point. It is feasible to estimate the performance directly by getting some samples and calculating the product of average encoding lengths, since the error is very likely within 10%. Buffers composed of 4-input LUTs with depth 16 are also sufficiently powerful, making the duty ratio close to 80%.

VI. CONCLUSIONS

Our software/hardware co-design achieves a better trade-off, that is, sacrificing some bit-sparsity revelation in exchange for the alleviation of excessive treatment, the balance of workloads, and the accuracy of calculations. The LUT-friendliness is embodied in two main aspects: one is the higher information density of encoding for better LUT-based transmission and calculation, and the other is the sufficient functionality of LUTRAMs to address multiplication time uncertainty. Enhancements in communication strategies achieved a temporal utilization increase of up to 2.79× compared to the naive strategy, at a tolerable cost of 2 CLBs per unit. The proposed FPGA-based implementation achieves up to 1.74× area efficiency under general-purposed integer quantization workloads, demonstrating compatibility with the design paradigm.

REFERENCES

[1] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 382–394.

[2] M. Mahmoud, K. Siu, and A. Moshovos, "Diffy: a déjà vu-free differential deep neural network accelerator," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 134–147.

[3] L. Cavigelli, G. Rutishauser, and L. Benini, "Ebpc: Extended bit-plane compression for deep neural network inference and training accelerators," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 4, pp. 723–734, 2019.

[4] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," in *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 304–317.

[5] E. Park, D. Kim, and S. Yoo, "Energy-efficient neural network accelerator based on outlier-aware low-precision computation," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 688–698.

[6] F. Liu, N. Yang, H. Li, Z. Wang, Z. Song, S. Pei, and L. Jiang, "Spark: Scalable and precision-aware acceleration of neural networks via efficient encoding," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 1029–1042.

[7] H. Kung, B. McDanel, and S. Qian Zhang, "Term revealing: Furthering quantization at run time on quantized dnns," *arXiv e-prints*, pp. arXiv–2007, 2020.

[8] X. Dai, Y. Chen, and M. S. Abdelfattah, "Kratos: An fpga benchmark for unrolled dnns with fine-grained sparsity and mixed precision," in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*, 2024, pp. 156–163.

[9] K. Sun, M. Koch, Z. Wang, S. Jovanovic, H. Rabah, and S. Simon, "An fpga-based residual recurrent neural network for real-time video super-resolution," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 4, pp. 1739–1750, 2022.

[10] S.-E. Chang, Y. Li, M. Sun, R. Shi, H. K.-H. So, X. Qian, Y. Wang, and X. Lin, "Mix and match: A novel fpga-centric deep neural network quantization framework," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 208–220.

[11] Y. Chen, Y. Xie, L. Song, F. Chen, and T. Tang, "A survey of accelerator architectures for deep neural networks," *Engineering*, vol. 6, no. 3, pp. 264–274, 2020.

[12] (2021) Achieving fp32 accuracy for int8 inference using quantization aware training with nvidia tensorrt. [Online]. Available: https://developer.nvidia.com/blog/achieving-fp32-accuracy-for-int8-inference-using-quantization-aware-training-with-tensorrt/

[13] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.

[14] F. Liu, W. Zhao, Z. Wang, Y. Chen, Z. He, N. Jing, X. Liang, and L. Jiang, "Ebsp: evolving bit sparsity patterns for hardware-friendly inference of quantized deep neural networks," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 259–264. [Online]. Available: https://doi.org/10.1145/3489517.3530660

[15] J. Kim, M. Sullivan, E. Choukse, and M. Erez, "Bit-plane compression: Transforming data for better compression in many-core architectures," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 329–340.

[16] L. Cavigelli, G. Rutishauser, and L. Benini, "Ebpc: Extended bit-plane compression for deep neural network inference and training accelerators," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 4, pp. 723–734, 2019.

[17] Y.-C. Lo and R.-S. Liu, "Bit-serial cache: Exploiting input bit vector repetition to accelerate bit-serial inference," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023.

[18] S. Q. Zhang, B. McDanel, H. T. Kung, and X. Dong, "Training for multi-resolution inference using reusable quantization terms," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 845–860. [Online]. Available: https://doi.org/10.1145/3445814.3446741

[19] X. Zhao, Y. Wang, C. Liu, C. Shi, K. Tu, and L. Zhang, "Bitpruner: Network pruning for bit-serial accelerators," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020.

[20] C. Sakr and N. Shanbhag, "An analytical method to determine minimum per-layer precision of deep neural networks," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 1090–1094.

[21] C. Tang, K. Ouyang, Z. Wang, Y. Zhu, W. Ji, Y. Wang, and W. Zhu, "Mixed-precision neural network quantization via learned layer-wise importance," in *Computer Vision – ECCV 2022*, S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds. Cham: Springer Nature Switzerland, 2022, pp. 259–275.

[22] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, J. K. Kim, V. Chandra, and H. Esmaeilzadeh, "Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 764–775.

[23] S. Ryu, H. Kim, W. Yi, and J.-J. Kim, "Bitblade: Area and energy-efficient precision-scalable neural network accelerator with bitwise summation," in *2019 56th ACM/IEEE Design Automation Conference (DAC)*, 2019.

[24] S. Lee and Y. Kim, "Booth fusion: Efficient bit fusion multiplier with booth encoding," in *2020 International SoC Design Conference (ISOCC)*, 2020, pp. 73–74.

[25] (2006) Fpga architecture white paper. [Online]. Available: https://cdrdv2-public.intel.com/771003/wp-01003.pdf/

[26] N. Brunie, F. de Dinechin, M. Istoan, G. Sergent, K. Illyes, and B. Popa, "Arithmetic core generation using bit heaps," in *2013 23rd International Conference on Field programmable Logic and Applications*, 2013.

[27] M. Kumm, S. Abbas, and P. Zipf, "An efficient softcore multiplier architecture for xilinx fpgas," in *2015 IEEE 22nd Symposium on Computer Arithmetic*, 2015, pp. 18–25.

[28] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "Sparten: A sparse tensor accelerator for convolutional neural networks," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, pp. 151–165. [Online]. Available: https://doi.org/10.1145/3352460.3358291